

Generics

هناك generics فكرتها انك من تحديد ال Type من الاول يعني انك لو عاوز تضيف في ال Array في ال Data Type مختلف في العادي هتستخدم union |
وتتالي نفهم ال generics كده اننا هنتكلم ان هتستعمل ال generics في ال Array

Function <returnType> <GenericsType> (val: GenericsType)

GenericsType {

return val }

طبعا كذا نص
الرجوع اليه
الرجوع اليه

Console.log (returnType <number> (200))
" " (returnType <String> ("Faragello"))
" " (returnType <boolean> (true))
" " (returnType <number[]> ([1,2,3,4]))

وبالشكل ده اصبحت ال code قابلة للاستخدام وتنقسم في ال Array
يكون الصاجه اني اتعمل مع Any

generics multiple types

const returnType ArrowSyntax = <T> (val: T): T =>

Console.log (returnType ArrowSyntax (number) (200))

general

Pick

هياخذ مجموعة محددة من ال التي انت كاتبها مش كلهم

interface User {

id: number,
name: string
age: number
email: string }

كده انا هتستعمل على ال name, email

type. userPreview = Pick <User, "name" | "email">

const u: userPreview = {

name: "Faragello",
email: "atefkerolos21@gmail.com" }

هنا هتستعمل انك حددت البيانات التي بتستعمل عليها

Record

هيفضل ان كل حاجة انت حاطها موجودة وان القيم نوعها متساوي

هنا في ال multiple فترتها ان باخذ parameter على

Function multipleTypes <T,S> (valueone, valueTwo) : String

return The First Value \$ {valueOne} and
Second Value \$ {valueTwo}

Console.log(multipleTypes('string', number) (1000))

فكرة ال parameter ان آتني كل الخصائص اختيارية وده سن
د اني بيصت قبل ال parameter ال parameter

Type User = { username: String,

age: number,

email: String }

const u: User = {
username: "Farghalla",

لما كنت بعمل كده في ال parameter كانت شكلها اني معرفش
احط على اول واحد لكن في ال parameter آفد انضدو على اى
اختيار

Interface Extend

نفس فکره فنا هم کن اخلا interface باخذ داتا و جمع علوم
interface بدون intersection یعنی بخشی interface جدید بناء على
موجود

```
interface Admin extends User {  
    protect: boolean | string;  
}
```

فنا گده و لا Admin لازم يدخل كل بيانات ال user + Admin

+ فنا أكبر فرق هيظهر بيت ال interface و Type Alias هي ال
Extend

Class Type

تعالى نزود في User
SayWelcome: () => String

ونزود في النتيجة الثانية
sayWelcome: () => {
 return 'welcome \${user.id}'
}

getDouble (num: number): number

getDouble(n) {
 return n * 2
}

افكار بست عنه البار مشرو ال Arrow

Interface Reopen

هنا الفكرة انه يفتح ال Interface ويضيف عليه ثانية
interface Settings { theme: boolean, font: string }
interface Settings { sidebar?: boolean }
let userInterface: Settings = { Theme: true,
 Font: "open sans", Sidebar: false }

خس باله هنا ممكن اكشفه اولاً لان optional

PreServeConstEnums

هو لو انت عال في ال Enums وتقدر ال Const
اي ال Compile يقدرك ال Enum ويخليك تستدلك ال Const
كل واحد واحد = Predefined Constants
من حد ال Const عند التحويل لـ كذا وده بقى ال
ال Outils هي توضح ال Enum لكن بس زور وفي ال Const
وقدر ال Preserve ممكن و ال Enum المستطاهم راحل بعض يعني اجيب
بنا سالت هه ال Enum الى فوقيها

Type Assertions

قائمة مستند + عناصره واحد DOMList
إذا كان الـ document.getElementByID() "myImg"
let myImg = document.getElementById("myImg")
CanSOLE.log(myImg.src) as HTMLImageElement

هنا سنضيف غير ال Attribute الى اربع الصور

Intersection &

Intersection & Union

Intersection: $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$

Union: $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$

Example:

Let $A = \{1, 2, 3, 4, 5\}$ and $B = \{3, 4, 5, 6, 7\}$

Then $A \cap B = \{3, 4, 5\}$ and $A \cup B = \{1, 2, 3, 4, 5, 6, 7\}$

(Intersection with interface)

```
interface Admin {  
  name: String,  
  privileges: String[]  
}
```

```
interface Employee {  
  name: String,  
  startDate: Date  
}
```

```
Type ElevatedEmployee = Admin & Employee
```

بالشكل ده الـ Interface جوه بعض

Type Annotations with object

let myObj:

```
{ username: String,  
  id: number,  
  hire: boolean } = {  
  username: "Faragello",  
  id: 01212092100,  
  hire: true  
}
```

skills = { one: String,
 two: String }

skills = { one: "summing",
 two: "Football" }

لوعاوز احدث البيانات
myObj.userName = "Ader"

readonly:

دستخداي عيشان محدث يقر بدل على البيانات

const arr: readonly number[] = [1, 2, 3]
 مستحيل عيشان readonly
 arr.push(4) →
 مفيدة لوضع Data ومثل عاوز د بدل عليها

Interface

interface User {

id: number,

username: string

country: string }

let use: User = { id: 233, ... }

console.log(user)

صفتها البياناتي ال Type Alias

interface method

interface User {

id: number,

sayHello(): string }

let user: User = {

id: 100,

sayHello() }

union

number

Unreachable Code

اولا الفرق بين never و never

Function هنا بمافيقص Error سواء throw او infinite
 return اي شيء في function

Unreachable هو الذي عيشان لو عاوز انخلي في الاختيارات
 التي على الاكوار التي مش هتتقد مثلا بعد throw او throw
 وانت مش عاوزه يتبدل مشكلة ال unreachable ممكن تكون
 عامل حسابية عليها اصل الهم هتتقدوا True

Enums

بصيا مقص ال Enum بتخبر في اوقات ثابتة او حلقه فيهم

enum Directions { up=1, down=2, left=3, right=4 }

const Kids = 15

const Easy = 9

const medium = 7

const hard = 3

enum Level {

Kids = 15

Easy = 9

medium = 7

hard = 3 }

let lvl: string = "Easy"

هنا شكل مقص ال Enum
 if (lvl === "Easy")

Type Annotation with arrow

```
const addWithArrow = (num1: number, num2: number): number {  
  return num1 + num2;  
}
```

هنا عشان ال return

Type Alias

هنا فكرتها اننا لو عاوز اعمل شي كده نوع ليا أنا بدل ما كل مرة مثلا نعمل String | number فيخليها واحدة واحدة

Type sn = String | number

واستعملهم في ال type script

Type Alias Advanced

Type Buttons = ?

up: string,

down: string,

right: string }

function getActions(btns: Buttons) {

console.log('up', btns.up);

getActions({ up: 'up', down: 'down', right: 'right' })

```
Function sum (... numbers, number[]) {
  return numbers.reduce((acc, cur) => acc + cur, 0)
} console.log(sum(1, 2, 3)) // 6
```

هنا الـ `reduce` كأنها هتختلف لحد ما تختلف أنت كاتب إن البداية 0
 القيمة الأولى $acc = 0$ / $cur = 1$ / $acc = 1$ وهكذا $acc = 1$ / $cur = 2$ / $acc = 3$ وهكذا
 في القيمة الثالثة المهم إن أحنا وهما طبيعة الـ `rest`

3] **scope** Variable declaration
 Function local scope
 انت ممكن تكون عامل
 في مضمون انه يطلعك تنبيه كده انك مستخدم ده لكن عامل
 بيته اي حاجه في انت هنا مضمون تفعل كالمستخدم كده
 هيطلعك ~~error~~ على Variable او local وده المطلوب
 رسالة تنبيه

optional and default Parameters (?)

دي بالمختصر المفيد معناها يا ما هتجيب البيانات فلما X
 يعني بقت اختياري ممكن يثبتت اولاً
 مثال

```
1] Function great(name?: String) {  
  if (name) => console.log("Hello" + name)  
  else => console.log("Hi")  
}
```

Rest Parameter

نفس فكرة ال rest في الجافا سكربت لكن في ال TypeScript بعد ذلك
 نوع البيانات الي محتاجها تكون بالداخل

```
function addAll(...nums: number[]): number {  
  let result = 0  
  return result {  
    nums.forEach((num) => result += num)  
    return result  
  }  
  console.log(addAll(10, 20, 30, 200, {true}))  
}
```